Assignment 1: Naïve Bayes Classifier

Jake Matthews

October 25th 2026

##

Student Name: Jake Matthews

Course Code: CSCI 460 -- Fall 2026

Assignment Due Date: 10/25/26

GitHub Link: https://github.com/matthewsj17/Winthrop-CSCI460Fall26/tree/main/Assignment1

##

Dataset Overview

The data used in this project is the Bank Marketing dataset downloaded from the UCI Machine Learning Repository. In its original unaltered form, the data contains 17 dimensions consisting of 45,211 individual instances. The dataset has the following predictor variables: age, job, marital, education, default, balance, housing, loan, contact, day_of_week, month, duration, campaign, pdays, previous, poutcome. These predictor variables are utilized in attempt to predict a classification of whether or not the client has a term deposit.

No true issues were found in the data cleaning processes. A fairly straight-forward approach was taken in order to ensure the data was in correct formatting for the Naive Bayes classification model. The steps taken were:

1. Check for missing values / NaNs
2. Encode all categorical variables into integers

Since no missing values or NaNs were found in the dataset, removing data wasn't necessary. Categorical values were encoded into integers using Pandas map functionality.

Experiment

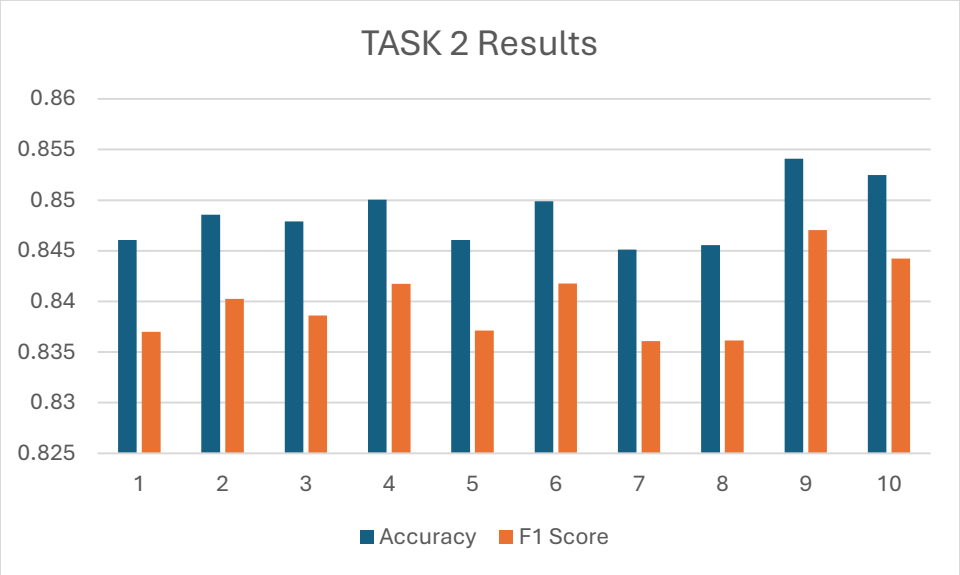
The main objective of this experiment was to document potential changes in model performance due to different amounts of training data. This was done in two ways. First, a fairly normal data split was taken, 70% Training, 30% Testing. This model was trained on the data provided ten different times, with both accuracy and F1 scores recorded. This information can be seen below in the results section.

After this, differing levels of training data were used and results were recorded. Each level of data-splitting was trained, tested, and evaluated 10 times each on random seeds, ensuring the results weren't influenced by default sci-kit learn values. The distribution of training data was as follows: [.10, .20, .30, .40, .50, .60, .70, .80, .90].

Results

The following results are from the initial test of 70/30 Train/Test split using Sci-kit learns NBGaussian model.

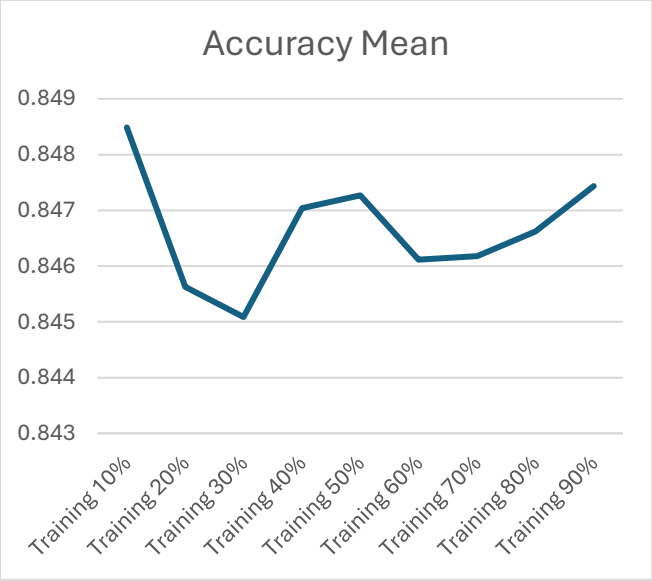
Testing Iteration	**Accuracy**	**F1 Score**
0	0.846063108	0.836994542
1	0.848569743	0.840260249
2	0.847906222	0.838617384
3	0.850044235	0.841728083
4	0.846063108	0.8371409
5	0.849896786	0.841777548
6	0.845104689	0.836082644
7	0.845547036	0.836142946
8	0.854099086	0.847039002
9	0.852477145	0.844224398
Mean	**0.8486**	**0.84**
Standard Deviation	**0.003054**	**0.003696**

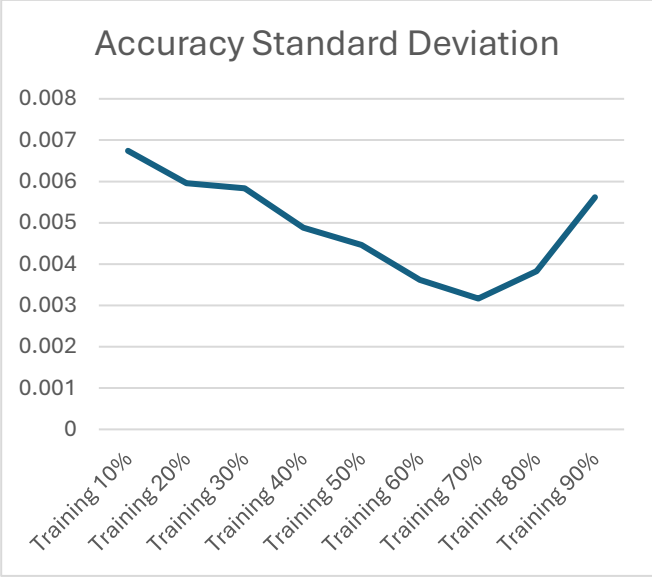

TASK 2 Results
0.86
0.855
0.85
0.845
0.84
0.835
0.83
0.825
1
2
3
4
5
6
7
8
9
10
Accuracy
F1 Score

The following tables and charts are from TASK 3.

ACCURACY

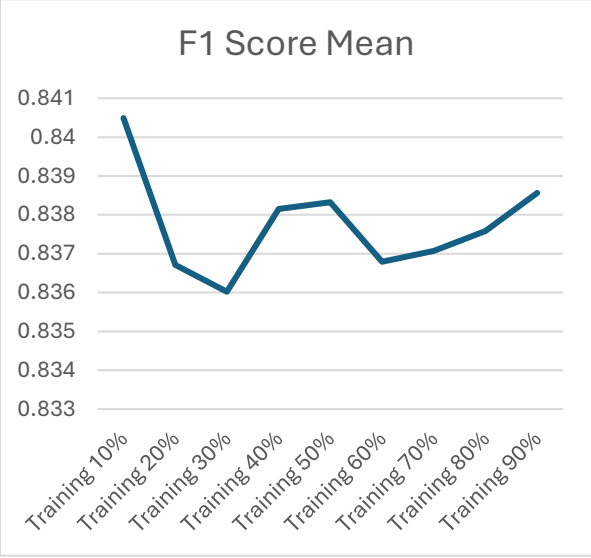
Testing Iteration	Training 10%	Training 20%	Training 30%	Training 40%	Training 50%	Training 60%	Training 70%	Training 80%	Training 90%
0	0.853649545	0.852719179	0.847446916	0.846868434	0.848933911	0.845839093	0.844072545	0.842198385	0.838567006
1	0.846399607	0.845115983	0.845740647	0.845172706	0.848226135	0.845341443	0.8439988 2	0.843967 71	0.838345865
2	0.847800442	0.850341453	0.849247978	0.847716297	0.846456693	0.843627315	0.844809791	0.841203141	0.849181778
3	0.847628 41	0.843650 64	0.839863498	0.842260478	0.845837388	0.849986176	0.848790917	0.849718014	0.850066342
4	0.847997051	0.836296276	0.844508342	0.846979025	0.847651066	0.847055571	0.845841935	0.842861882	0.849624 6
5	0.832686164	0.840968785	0.846909757	0.847531979	0.850305229	0.849433232	0.847979947	0.849939179	0.850729766
6	0.843130 9	0.8359645	0.831237361	0.839274524	0.840219411	0.840309649	0.841639634	0.846621696	0.845201238
7	0.852813959	0.848350798	0.844824317	0.847200206	0.844908431	0.843903788	0.849159 4	0.849607431	0.854489164
8	0.856819857	0.852359756	0.854714358	0.859291481	0.857559 4	0.852972076	0.852477145	0.853477828	0.854931446
9	0.855935119	0.850507341	0.846372599	0.848121797	0.842563921	0.842687 1	0.843040401	0.846621696	0.843210969

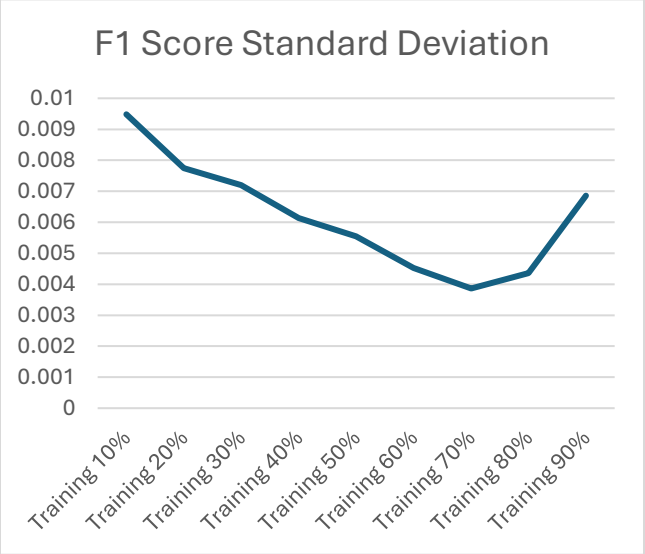




F1 SCORE

Testing Iteration	Training 10%	Training 20%	Training 30%	Training 40%	Training 50%	Training 60%	Training 70%	Training 80%	Training 90%
0	0.846957967	0.846650863	0.839227112	0.838269517	0.840848739	0.836757366	0.834695502	0.832217766	0.827294241
1	0.839531324	0.836717155	0.836650599	0.835728345	0.839173183	0.835849175	0.833996575	0.834271555	0.827442224
2	0.840186895	0.842497207	0.841055509	0.838996338	0.837263497	0.833866759	0.836218428	0.832258235	0.842084455
3	0.838477612	0.834510123	0.829672772	0.832912644	0.836958116	0.841831388	0.840361246	0.840007154	0.840316555
4	0.839511516	0.824803808	0.835402594	0.838055995	0.838955094	0.837403343	0.836141421	0.833097009	0.840594707
5	0.817346302	0.829830321	0.838019586	0.838823415	0.842143641	0.841204218	0.839110906	0.841421287	0.842407921
6	0.833607134	0.8239512 4	0.818529098	0.827358039	0.828945646	0.829221822	0.831689262	0.837739111	0.836258355
7	0.847191928	0.841872294	0.837227534	0.839196319	0.836271393	0.834301505	0.841358403	0.841979871	0.847021547
8	0.850587924	0.844170058	0.847368 4	0.853084972	0.850612223	0.845022 2	0.844310249	0.845416 7	0.848015275
9	0.851511265	0.842111668	0.837059326	0.839071112	0.832080038	0.832490137	0.832848285	0.837390678	0.834198084





Interpretation

It seems as though the experiment has revealed that in the specific cases of the data and classification model used, differences in training data usage don't really affect the performance of the classification model. Throughout all tests, the gaussian Naïve Bayes model achieved roughly 0.84 Accuracy and 0.83 F1 Score.

It is interesting how it seems that a train/test split of 70/30 offers the lowest standard deviation on average. This seems to allude to the model being most consistent at prediction around this split. For this reason I would consider 70/30 to be the sweet spot for this model, just because of the consistently throughout all iterations.

There are many possible limitations when it comes to this experimental design in interpreting the effects of train/test split. Many of them are listed below, although I am sure there are many more.

- Experiment only considers the UCI Bank Marketing dataset.
- Experiment doesn't consider other classification models/methods, just Gaussian Naïve Bayes.
- Experiment exclusively measures Accuracy and F1 Score, rather than a more comprehensive, complex measure like Receiver Operator Characteristic Curves.
- Experiment doesn't consider the unbalanced distributions of predicted classes.
    - In the data, *'no'* occurs 39922 times, while *'yes'* occurs 5289 times.

Rectifying these experimental shortcomings could offer a more complete picture on how Train/Test split distribution effects a given model's performance.